# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE NAME: Computer Vision with OpenCV COURSE CODE: DSA02**

**COURSE OUTCOME COVERED**

***CO5:*** *Apply computer vision techniques to real-world applications such as face recognition, tracking, medical imaging, and autonomous systems. (BL3)*

*Assessment Tool Weightage: 50%*

**QUESTIONS: 5 Total Marks:50** Annexure A

# Constraint-Based Problem Solving

***Code of Conduct:***

*I Cheredla jasvina/192324026 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

**RUBRICS FOR CALCULATION**

| Criteria | Weightage | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) | Marks |
|---|---|---|---|---|---|---|
| Problem Understanding | 25% | Clearly defines the problem and identifies all relevant constraints accurately | Identifies most constraints with minor gaps | Partial understanding; misses key constraints | Fails to identify constraints correctly | |
| Constraint Analysis | 25% | Analyzes constraints effectively and prioritizes them logically | Provides reasonable analysis with some prioritization | Limited analysis; weak prioritization | No meaningful analysis or prioritization | |
| Solution Development | 25% | Develops optimal and feasible solutions satisfying all constraints | Develops workable solutions with minor limitations | Solutions partially satisfy constraints | Solutions do not meet constraints | |
| Application of Concepts | 15% | Effectively applies appropriate concepts, methods, or tools | Applies concepts with minor errors | Limited application; requires guidance | Unable to apply relevant concepts | |

| | | | | | | |
|---|---|---|---|---|---|---|
| Justification | 10% | Provides strong justification for decisions with logical reasoning | Provides justification with some clarity | Weak or unclear justification | No justification provided | |

Annexure A

# Constraint-Based Problem Solving

**1. Secure Face Recognition in Low-Light Environment**

**1.1 How would you adapt Eigenface-based recognition to handle varying illumination and orientation?**

Eigenfaces are sensitive to changes in lighting and face orientation. To improve performance:

- Apply illumination normalization before PCA.
- Use histogram equalization or CLAHE to improve low-light images.
- Train the Eigenface database with faces captured under different lighting conditions and orientations.
- For ±30° orientation, use face alignment and landmark detection to rotate/normalize the face.
- Generate or include multiple views of each person's face, such as frontal, left, and right views.
- Use PCA after alignment so that Eigenfaces represent more consistent facial features.
- Set an appropriate recognition threshold to reject unknown faces.

**Basic flow:**

Input → Low-light enhancement → Face detection → Face alignment → Resize/normalize → PCA/Eigenfaces → Distance comparison → Recognition

**2 Suggest preprocessing steps to improve recognition under low-light conditions.**

**Suitable preprocessing steps are:**

- Convert the image to grayscale.
- Remove noise using a Gaussian or median filter.
- Apply CLAHE/histogram equalization to improve contrast.
- Perform gamma correction to brighten dark regions.
- Normalize pixel intensity.
- Detect facial landmarks.
- Align the face according to the eyes/nose position.
- Resize all faces to a fixed resolution.
- Optionally use illumination-invariant filtering to reduce strong shadows.
- These steps make the input more consistent before Eigenface extraction.

**3 Discuss strategies to optimize computation for real-time performance on constrained hardware.**

**To achieve recognition in less than 200 ms:**

- Use a small number of principal components instead of all Eigenfaces.

- Resize faces to a small fixed resolution.
- Restrict face detection to a region of interest where appropriate.
- Perform face detection less frequently and track faces between detections.
- Use optimized matrix operations and fixed-point/quantized computation where suitable.
- Store precomputed Eigenface projections of database faces.
- Use a lightweight face detector such as a Haar/Viola-Jones cascade.
- Use hardware acceleration such as an embedded DSP/NPU if available.
- Process only the detected face region rather than the entire image.

**Main idea:** Reduce image size, reduce PCA dimensions, and avoid repeated calculations.

**2. Photo Album Organization**

**1. How would you use face recognition and clustering to group photos of the same individual?**

**The system can use the following pipeline:**

- Scan each image locally.
- Detect all faces.
- Align and crop each face.
- Extract a facial feature vector/embedding.
- Compare embeddings between faces.
- Group similar embeddings using clustering, such as DBSCAN or hierarchical clustering.
- Assign each cluster to an individual.
- Allow the user to provide a name for a cluster.
- Store the face representation and cluster information locally.
- For millions of images, use an efficient nearest-neighbor search/index instead of comparing every face against every other face.

**2. Propose a method to detect and separate images with occlusions or partial faces.**

**The system can calculate a face quality score based on:**

- Percentage of visible facial area.
- Landmark detection confidence.
- Face size.
- Blur level.
- Pose angle.
- Occlusion detection.
- If important landmarks such as both eyes, nose, and mouth are missing, the face can be classified as partial/occluded.
- The system can then:
- Put these images into a separate “uncertain/occluded” group.
- Avoid using poor-quality faces as primary references.
- Use multiple good-quality photos of the same person to improve clustering.

**3. Discuss trade-offs between accuracy and computational efficiency for local processing.**

| **Approach** | **Accuracy** | **Computation** | **Privacy** |
|---|---|---|---|
| Simple Eigenfaces | Moderate | Low | Excellent |
| Lightweight face embeddings | High | Moderate | Excellent |
| Large deep-learning model | Very high | High | Excellent if run locally |
| Cloud processing | High | Low on device | Lower |

- For a local photo organizer, a lightweight face-recognition model + efficient clustering/indexing provides a good balance.
- Processing can also be done in batches when the device is idle, reducing the need for real-time performance.

**3. Smart City Multi-Camera Surveillance**

**1. How would you design foreground–background separation to minimize false positives in crowded scenes?**

**A simple frame-difference method can produce many false positives in crowded environments. A better approach is:**

- Build an adaptive background model.
- Use a Gaussian Mixture Model (GMM) or similar adaptive model.
- Detect foreground regions.
- Apply morphological operations to remove small noise.
- Use pedestrian/object detection to verify detected regions.
- Combine motion information with appearance information.
- Use temporal consistency so that an object must persist for several frames before being accepted.
- This reduces false detections caused by shadows, illumination changes, and small movements.

**2 Explain how particle filters can be used to handle occlusion during tracking.**

**A particle filter represents the possible position and motion of a pedestrian using many particles.**

**During normal tracking:**

Prediction → Observation → Weight particles → Resample

**During occlusion:**

- The pedestrian may temporarily disappear from the camera.
- The particle filter continues predicting possible positions using the person's previous velocity and motion.
- Particles spread around likely positions.
- When the person becomes visible again, the observation updates the particle weights.
- The tracker then converges back to the actual pedestrian.
- Thus, particle filters are useful when observations are missing or uncertain.

**3.3 Propose an efficient multi-camera strategy for consistent pedestrian identification.**

Each camera should perform most processing locally to reduce network bandwidth.

**Proposed architecture:**

Camera → Local detection → Local tracking → Feature extraction → Compact metadata → Central/edge association

**Each camera sends only:**

- Pedestrian ID
- Timestamp
- Position
- Compact appearance feature
- Camera ID
- Instead of transmitting full video streams.
- A central edge server can match pedestrians across cameras using:
- Appearance features
- Clothing/color information

- Height/body shape
- Movement direction
- Camera topology
- Time of appearance

This provides consistent identification while reducing network traffic.

**4. Self-Driving Car**

**1. How would you implement road detection robust to shadows, rain, and nighttime illumination?**

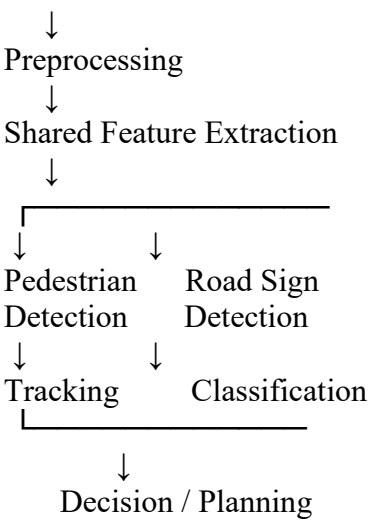
A robust road-detection system should combine multiple techniques.

**Pipeline:**

- Capture camera images.
- Perform image enhancement.
- Use color-space conversion such as HSV/Lab.
- Detect road boundaries.
- Use semantic segmentation to identify the drivable road region.
- Use temporal information from previous frames.
- Combine camera information with LiDAR/radar where available.
- Apply confidence estimation to handle poor visibility.
- For shadows, color and texture information can prevent dark regions from being incorrectly classified as non-road.
- For rain, denoising and temporal filtering can reduce the effect of raindrops and reflections.
- For nighttime, exposure/contrast enhancement and infrared or other sensors can improve detection.

**4.2 Propose a pipeline to simultaneously detect road signs and pedestrians efficiently.**

A practical pipeline is:




- A multi-task neural network can share early feature-extraction layers and have separate detection heads for pedestrians and signs.
- Tracking can then be applied between frames so that objects do not need to be detected from scratch every frame.

**3 Discuss trade-offs between deep learning-based detectors and classical computer vision methods under real-time constraints.**

| Feature | Classical CV | Deep Learning |
|---|---|---|
| Computation | Low | Usually higher |
| Training | Little/no training | Requires training data |
| Complex environments | Less robust | More robust |
| Shadows/weather | Limited | Generally better |
| Accuracy | Moderate | Usually higher |
| Hardware requirements | Low | Higher |
| Adaptability | Limited | High |

**Conclusion**

Classical methods such as Canny, Hough Transform, thresholding, and optical flow are computationally efficient and useful on constrained hardware.

Deep-learning methods such as CNN-based object detectors and segmentation networks provide better performance in complex conditions but require more computational resources.

For a real-time autonomous vehicle, a hybrid approach or lightweight optimized deep-learning model can provide a good balance between accuracy and speed.

**5. AR Surgical Assistance**

**5.1 How would you implement real-time organ segmentation under memory and latency constraints?**

- Because CT/MRI data are high-resolution, processing the complete 3D volume at once may exceed GPU memory.

**A suitable approach is:**

- Preprocess the CT/MRI volume.
- Divide it into smaller 3D patches/tiles.
- Process patches using a lightweight 3D segmentation network.
- Use reduced precision such as FP16 where supported.
- Combine the segmented patches.
- Apply post-processing to remove small false regions.
- Keep only the relevant organ region for AR rendering.
- Use temporal information to avoid recomputing unchanged regions.
- A lightweight 3D U-Net or compressed segmentation network can be used.

**5.2 Discuss methods to integrate segmentation results accurately in AR/MR overlays.**

The segmentation must be accurately aligned with the patient's actual anatomy.

**Important steps include:**

- Image-to-patient registration to align CT/MRI coordinates with the patient.
- Use anatomical landmarks or fiducial markers.
- Calibrate the AR headset/camera.
- Track patient and headset movement.
- Transform the segmentation from medical-image coordinates into AR coordinates.
- Continuously update the registration if the patient moves.
- Use depth information to improve spatial alignment.
- Display the segmentation with adjustable transparency.
- The key requirement is accurate spatial registration between the medical image and the real patient.

**3. Suggest strategies to reduce computational load while maintaining segmentation accuracy.**

- Useful strategies include:
- Use lightweight network architectures.
- Reduce the input resolution where clinically acceptable.
- Process only the region of interest (ROI).
- Use 2D/2.5D models when full 3D processing is unnecessary.
- Use model pruning.
- Use quantization.
- Use FP16/mixed-precision computation.
- Use patch-based processing.
- Reuse previous segmentation results instead of processing every frame from scratch.
- Use GPU/NPU acceleration.
- Perform expensive segmentation less frequently and use tracking/interpolation between updates.

**Overall approach**

The goal is to balance **accuracy, latency, and memory usage**:

ROI selection + lightweight model + patch processing + quantization + hardware acceleration + temporal reuse

This can help achieve the required <100 ms interaction latency while preserving clinically useful segmentation quality.